



05506026

Digital Image Processing

Witchaya Towongpaichayont
Computer Science, KMITL

CONTENTS



1

FEATURE EXTRACTION



FEATURE EXTRACTION

Background

- ▶ Image segmentations have left us with a number of regions in image. We must use feature extraction to convert from quantitative attributes (points, edgels, regions) into some meaningful information, which human can consume.
- ▶ An image must be extracted on how to be represented and described for further processing
- ▶ There are two choices to represent a region
 - ▶ External characteristics (boundary)
 - ▶ Internal characteristics (pixels comprising the region – colours, textures, etc)

Background

- ▶ Normally feature extraction consists of two parts
 - ▶ Feature detection
 - ▶ Feature Description
- ▶ On the other hand, the features we are interested in can be grouped into 3 principal categories.
 - ▶ Boundary
 - ▶ Region
 - ▶ Whole Image



Boundary Preprocessing

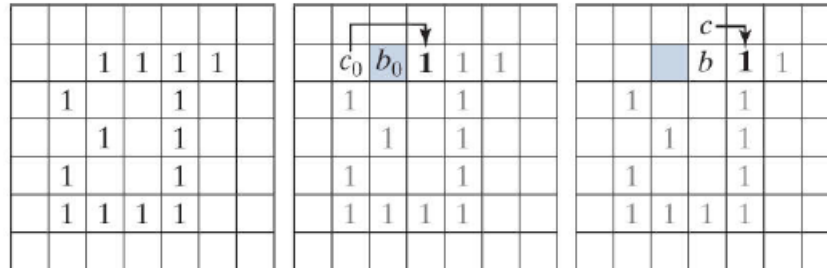
Boundary Following (Tracing)

- ▶ Before starting, we must know a couple of things about boundary we are discussing here
 1. We are only working on a binary image first (1 or 0 values)
 2. Image must be padded with a border of 0s
 3. We are discussing about a single boundary. For the whole image, we must extend the processing to each boundary individually.

Moore Boundary Tracing Algorithm

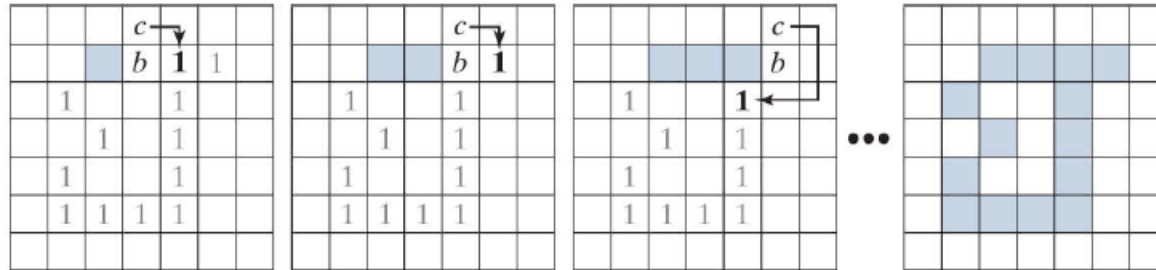
► The algorithm to trace the boundary of a 1-valued region is as follow.

1. Start with point b_0 at the **uppermost-leftmost point in the image that is labeled 1**
2. Denote by c_0 the west neighbour of b_0 . c_0 is always a background point. Starting from c_0 , examine 8-neighbours of b_0 clockwise starting from c_0 . Mark the **first** 1-valued point as b_1 and mark the point preceding b_1 as c_1



Moore Boundary Tracing Algorithm

3. Keep tracking in this fashion of b and c until $b = b_0$
4. The list of $b_0, b_1, \dots, b_n$ is the boundary points



- ▷ This algorithm can extract more complex outer boundary of the region as follow

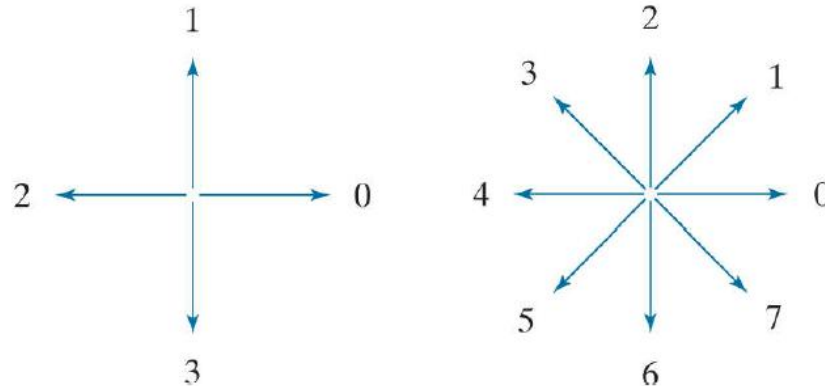
- ▷ The extracted boundaries are normally, but not always, one pixel thick
- ▷ If the objective is to find the inner boundaries (or hole) of the region, we can extract the holes and treat them as another region

Chain Codes

- ▶ Chain codes are used to represent a boundary by a connected sequence of straight-line segments of specified length and direction
- ▶ The Chain Codes are used to store the boundary which will reduce the amount of data significantly
- ▶ Note that we assume that all curve are closed and simple (not self intersecting)

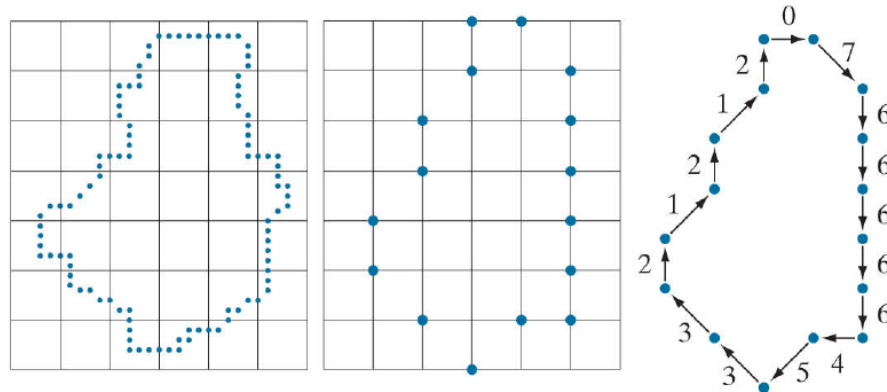
Freeman Chain Codes

- ▶ A Chain Code representation is based on 4- or 8-connectivity of the segments. This represents the direction of the boundaries



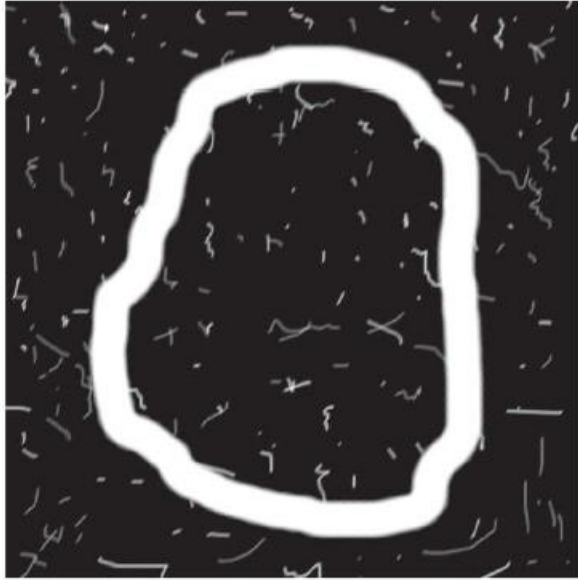
Freeman Chain Codes

- Therefore, a list of boundary points can be coded by the direction of the next boundary point as the example below



- This boundaries' chain code is 0 7 6 6 6 6 6 4 5 3 3 2 1 2 1 2

Freeman Chain Codes



Original image

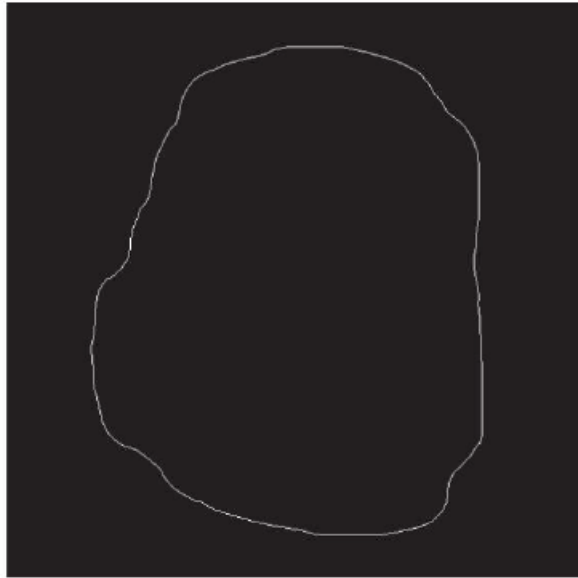


Smoothed image

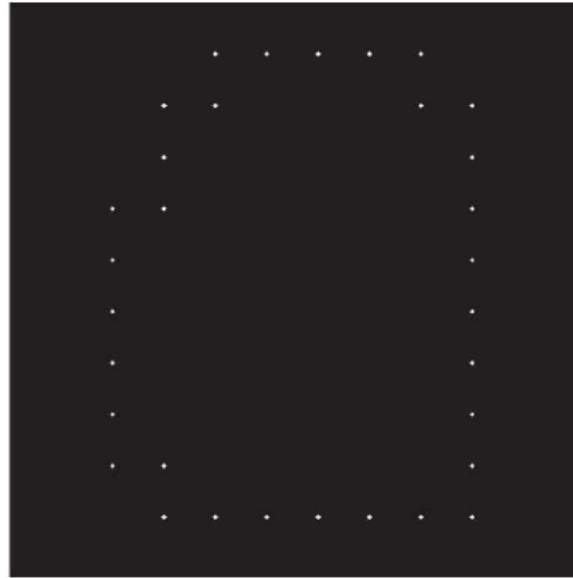


Thresholded using Otsu's

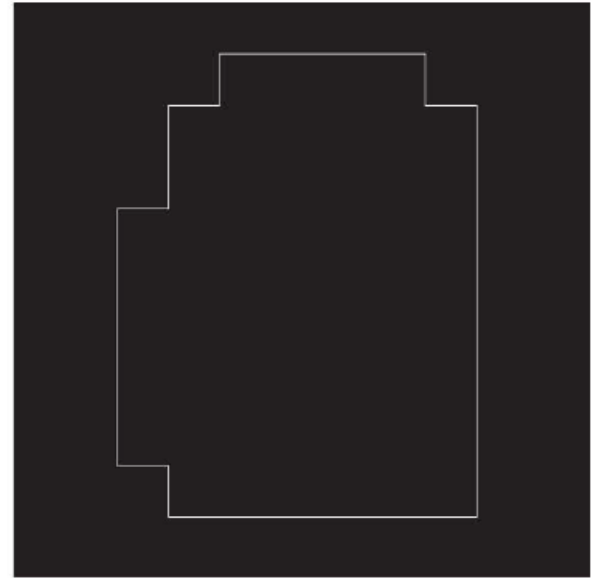
Freeman Chain Codes



Boundary extracted

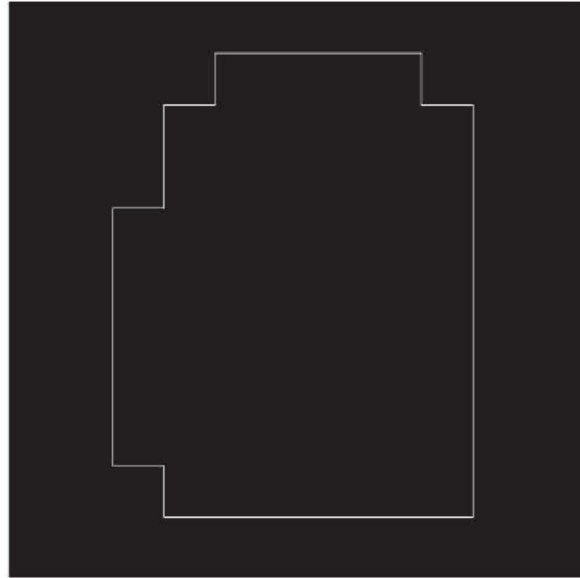


**Sampled points from extracted
boundary with 50 px apart**



Connected points

Freeman Chain Codes

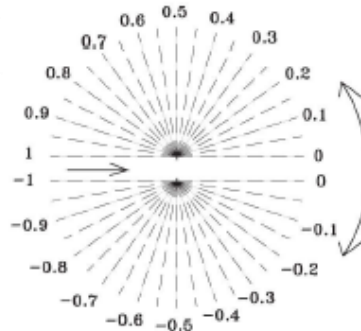


The chain code is

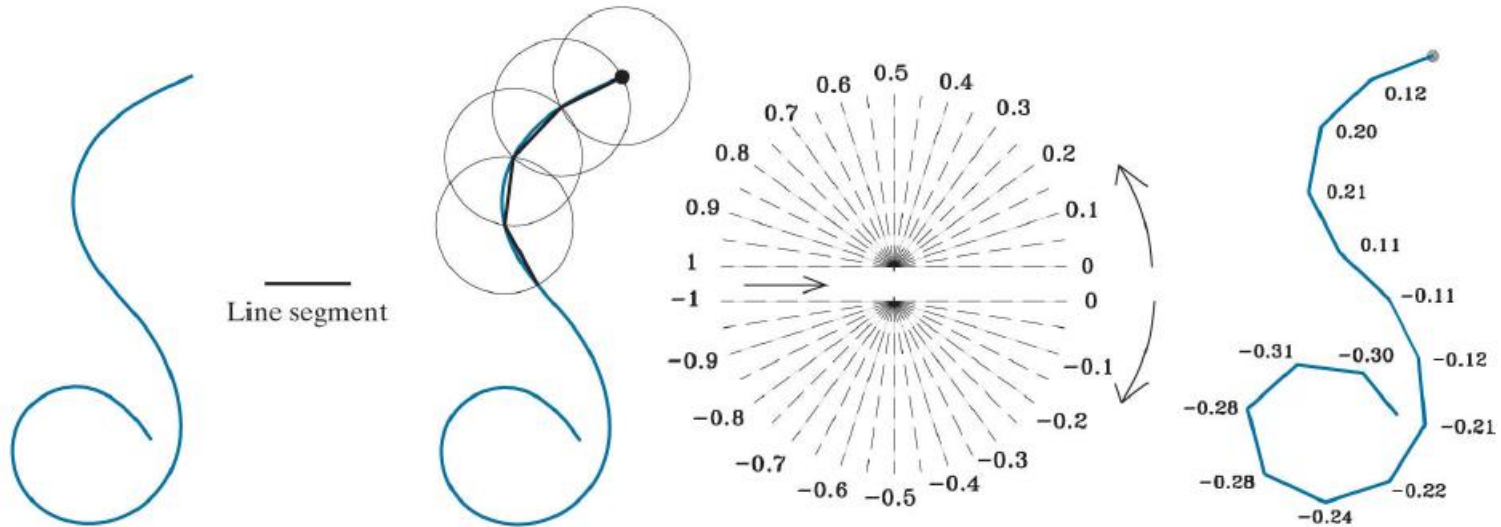
0 0 0 0 6 0 6 6 6 6 6 6 6 4 4 4 4 4 2 2 2 2 2 0 2 2 0 2

Slope Chain Codes

- ▶ Freeman Chain Codes need to resample a boundary which result some errors in some boundaries.
- ▶ Slope Chain Codes are an alternative approach which use the direction which is relative to the preceding point instead. The direction is normalised in the continuous interval $(-1, 1)$



Slope Chain Codes

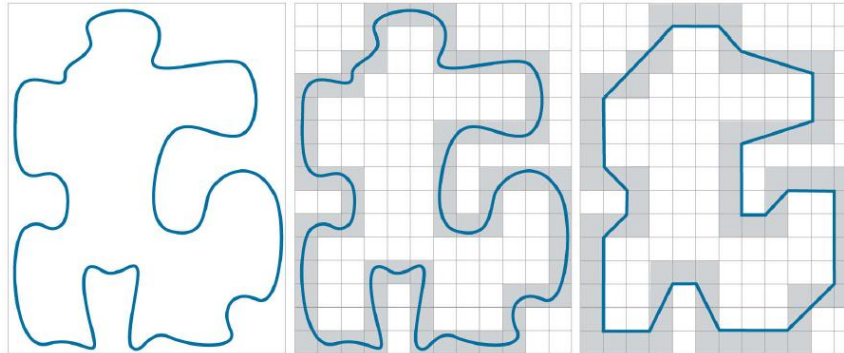


The chain code is

0.12, 0.20, 0.21, 0.11, - 0.11, - 0.12, - 0.21, - 0.22, - 0.24, - 0.28, - 0.28, - 0.31, - 0.30

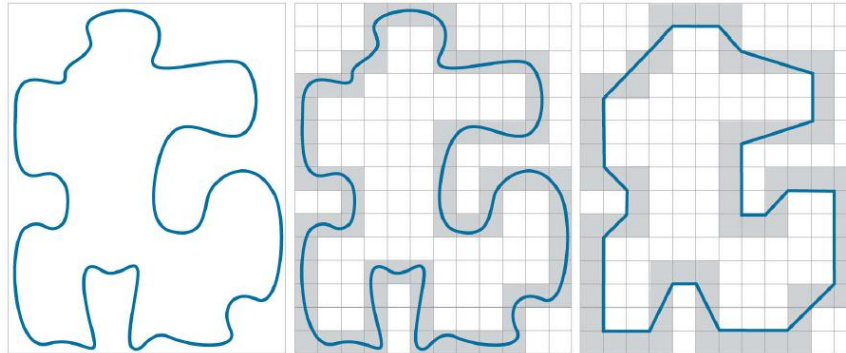
Minimum-Perimeter Polygons

- ▶ Minimum-Perimeter Polygons (MPP) is a technique of boundary approximations in which the boundary is approximated with arbitrary accuracy by a polygon
- ▶ Imagine the boundary as a rubber band which is contained in the gray cells of inner and outer wall of the region



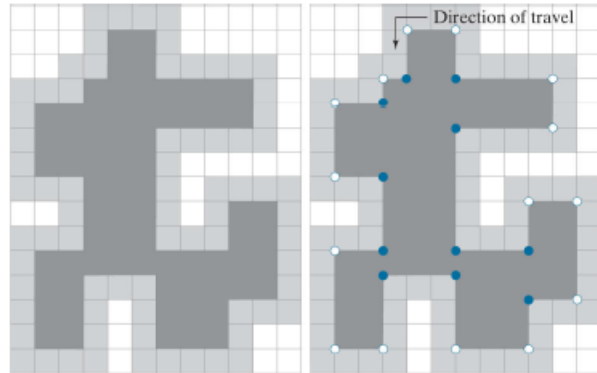
Minimum-Perimeter Polygons

- ▶ The rubber band can shrink which is constrained by the vertices of the inner and outer walls which produces the shape of polygon of minimum perimeter
- ▶ The accuracy of the approximation depends on the size of each square cell



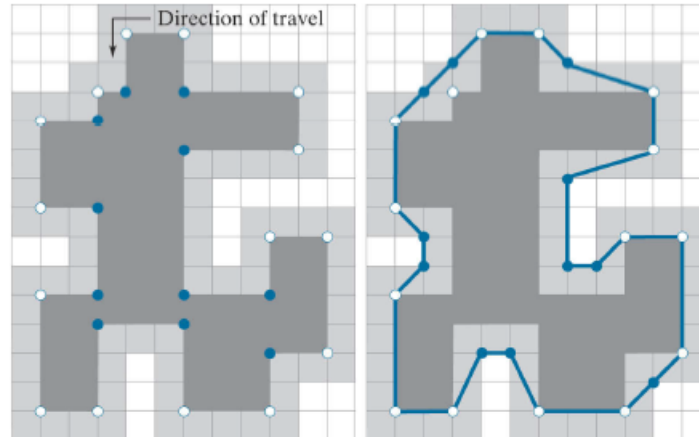
Minimum-Perimeter Polygons

- ▶ When we use a rubber band to travel within the walls, every turn we will encounter either a convex or a concave vertex (defined by the interior angle of the boundary). The convex vertex is shown below as a white dot and the concave one is the blue dot



Minimum-Perimeter Polygons

- ▶ If we put the rubber band into the wall, we will see that the vertices of the MPP coincide either with convex vertices in the inner wall or the mirrors of the concave vertices in the outer wall. This is the foundation of MPP algorithm



Minimum-Perimeter Polygons

- ▶ The algorithm to find MPP marks W as convex vertex and B as mirrored concave vertex.
- ▶ We also define that any three points $a = (a_x, a_y)$, $b = (b_x, b_y)$, and $c = (c_x, c_y)$ can be arranged in form

$$A = \begin{bmatrix} a_x & a_y & 1 \\ b_x & b_y & 1 \\ c_x & c_y & 1 \end{bmatrix}$$

- ▶ In which we can analyse as follow

$$\det(\mathbf{A}) = \begin{cases} > 0 & \text{if } (a, b, c) \text{ is a counterclockwise sequence} \\ 0 & \text{if the points are colinear} \\ < 0 & \text{if } (a, b, c) \text{ is a clockwise sequence} \end{cases}$$

Minimum-Perimeter Polygons

$$\det(\mathbf{A}) = \begin{cases} > 0 & \text{if } (a, b, c) \text{ is a counterclockwise sequence} \\ 0 & \text{if the points are colinear} \\ < 0 & \text{if } (a, b, c) \text{ is a clockwise sequence} \end{cases}$$

- ▶ For our convenience, we will define that $\text{sgn}(a, b, c) \equiv \det(A)$
- ▶ In the right-handed coordinate system where the origin is at the top left, going counterclockwise means point c lies on the POSITIVE side of a and b , while going clockwise means point c lies on the NEGATIVE side of a and b

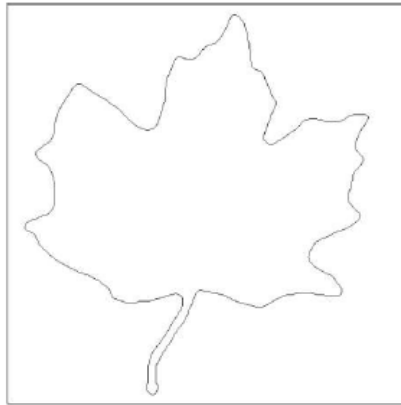
Minimum-Perimeter Polygons

- ▶ The algorithm uses two pointers: - W_c to observe white (convex) point and B_c to observe blue (mirrored concave) point.
- ▶ The algorithm starts by setting $W_c = B_c = V_0$ (First MPP vertex) then let V_L denote the last MPP vertex found and let V_k denote the current vertex being examined
 - ▶ If $\text{sgn}(V_L, W_c, V_k) > 0$, the next MPP vertex is W_c , and we let $V_L = W_c$ then set $W_c = B_c = V_L$
 - ▶ If $\text{sgn}(V_L, W_c, V_k) \leq 0$ or $\text{sgn}(V_L, B_c, V_k) \geq 0$, we set $W_c = V_k$ if V_k is convex or $B_c = V_k$ if V_k is mirrored concave
 - ▶ If $\text{sgn}(V_L, B_c, V_k) < 0$, the next MPP vertex is B_c , and we let $V_L = B_c$ then set $W_c = B_c = V_L$
- ▶ The algorithm stops when it reaches the first vertex again

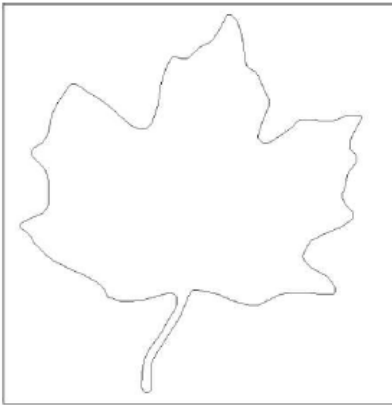
Original image



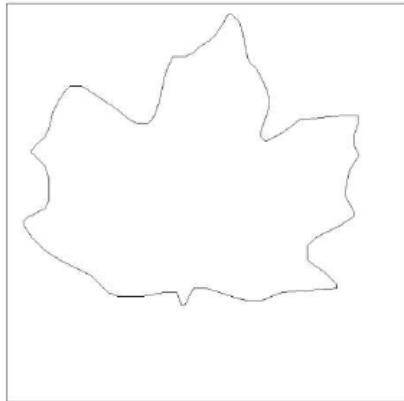
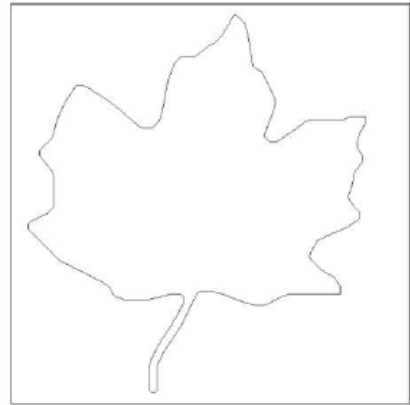
8-connected boundary



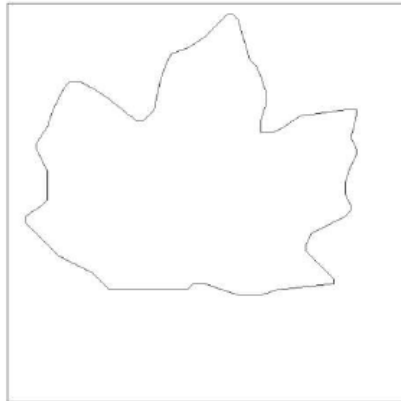
**MPP using 2x2 cell
(206 vertices)**



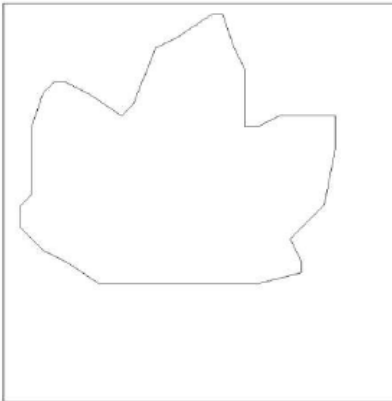
**MPP using 4x4 cell
(127 vertices)**



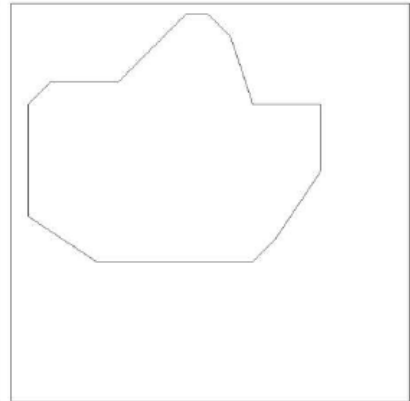
**MPP using 6x6 cell
(92 vertices)**



**MPP using 8x8 cell
(66 vertices)**



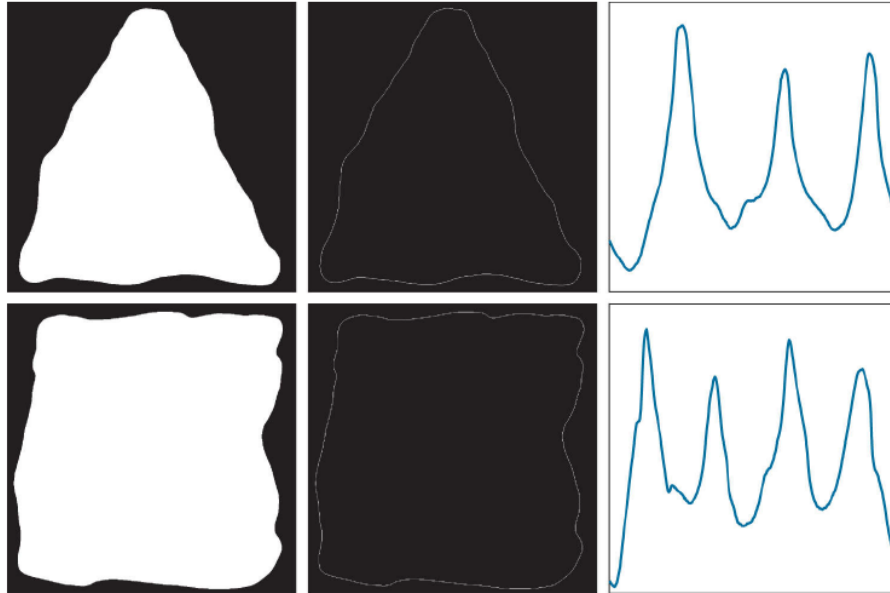
**MPP using 16x16 cell
(32 vertices)**



**MPP using 32x32 cell
(13 vertices)**

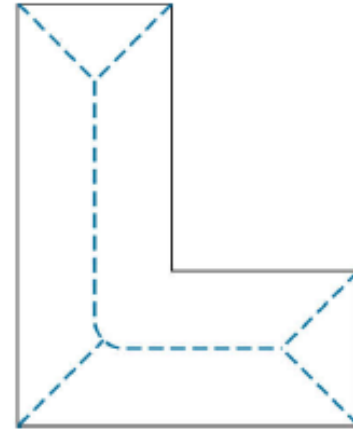
Other boundary representations

▶ 1-D Signatures



Other boundary representations

► Medial Axes



Other boundary representations

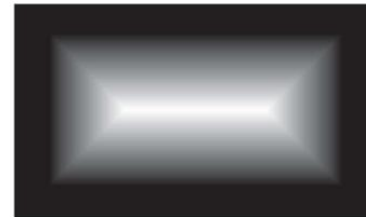
▷ Distance transform

0	0	0	0	0
0	1	1	1	0
0	1	1	1	0
0	0	0	0	0

1.41	1	1	1	1.41
1	0	0	0	1
1	0	0	0	1
1.41	1	1	1	1.41

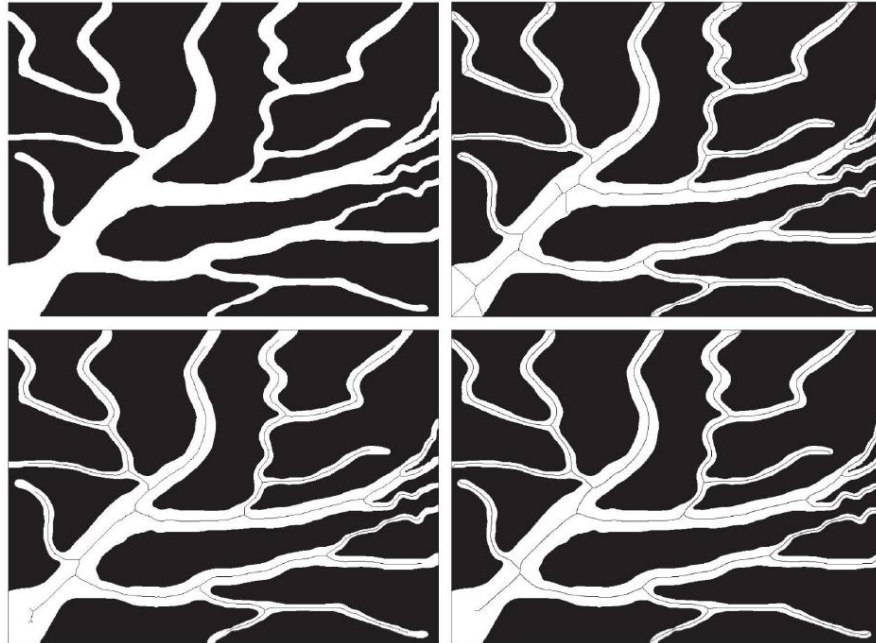
0	0	0	0	0	0	0	0	0
0	1	1	1	1	1	1	1	0
0	1	1	1	1	1	1	1	0
0	1	1	1	1	1	1	1	0
0	1	1	1	1	1	1	1	0
0	1	1	1	1	1	1	1	0
0	0	0	0	0	0	0	0	0

0	0	0	0	0	0	0	0	0
0	1	1	1	1	1	1	1	0
0	1	2	2	2	2	2	1	0
0	1	2	3	3	3	2	1	0
0	1	2	2	2	2	2	1	0
0	1	1	1	1	1	1	1	0
0	0	0	0	0	0	0	0	0



Other boundary representations

▷ Skeletons





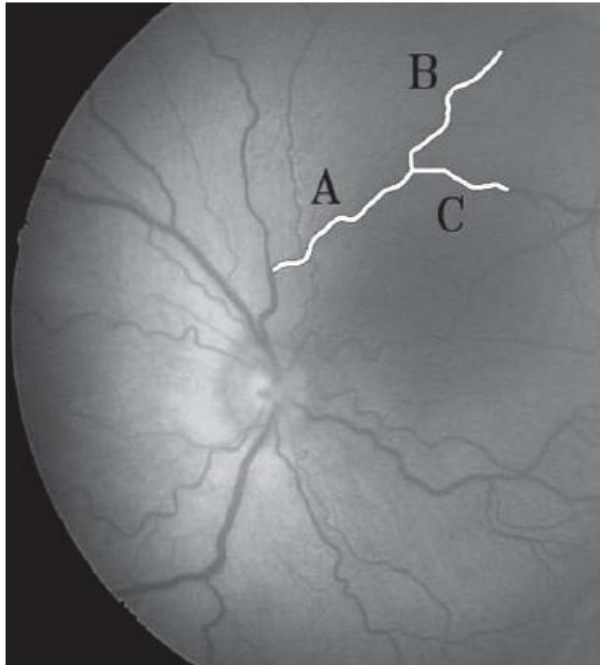
Boundary Feature Descriptors

Basic Boundary Descriptors

- ▶ One of simplest boundary descriptors is the **length** which is approximated by the number of pixels along the boundary.
- ▶ For a chain code, the length is
$$number_{vertical} + number_{horizontal} + \sqrt{2} \times number_{diagonal}$$
- ▶ The second basic descriptor is **curvature** which is the rate of change of slope. It can be easily obtained using slope chain code representation
- ▶ Another basic descriptor is **tortuosity** which is a measure of the twists and turns. It can also be easily obtained using slope chain code representation and calculate by

$$\tau = \sum_{i=1}^n |a_i|$$

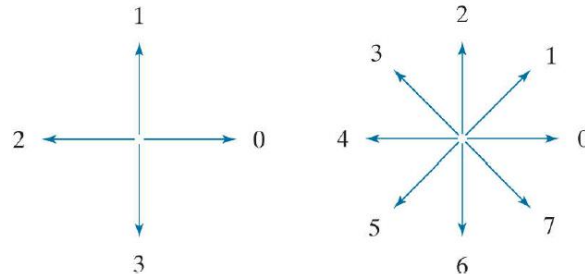
Basic Boundary Descriptors



Curve	n	τ
A	50	2.3770
B	50	2.5132
C	50	1.6285

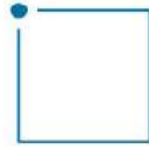
Shape Numbers

- ▶ The shape number of a Freeman chain-coded boundary, which based on the 4-directional code, is defined as the first difference of smallest magnitude
- ▶ The order of a shape number, n , is defined as the number of digits in its representation.
- ▶ The first difference is the difference between one direction to another counterclockwise



Shape Numbers

Order 4



Chain code: 0 3 2 1

Difference: 3 3 3 3

Shape no.: 3 3 3 3

Order 6



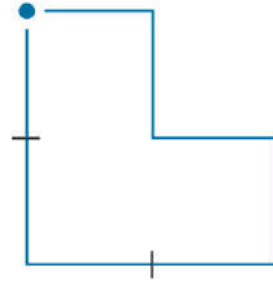
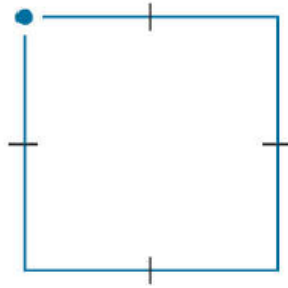
Chain code: 0 0 3 2 2 1

Difference: 3 0 3 3 0 3

Shape no.: 0 3 3 0 3 3

Shape Numbers

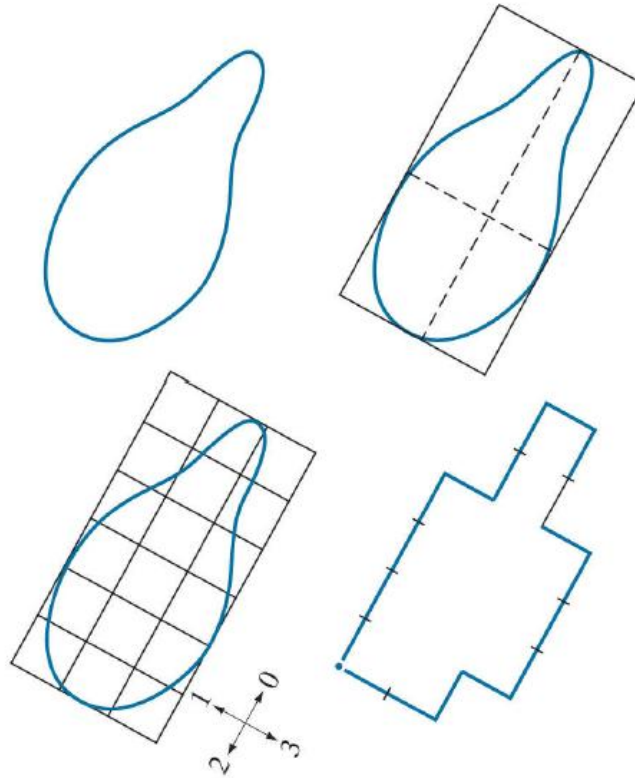
Order 8



Chain code: 0 0 3 3 2 2 1 1 0 3 0 3 2 2 1 1 0 0 0 3 2 2 2 1

Difference: 3 0 3 0 3 0 3 0 3 3 1 3 3 0 3 0 3 0 0 3 3 0 0 3

Shape no.: 0 3 0 3 0 3 0 3 0 3 0 3 3 1 3 3 0 0 3 3 0 0 3 3



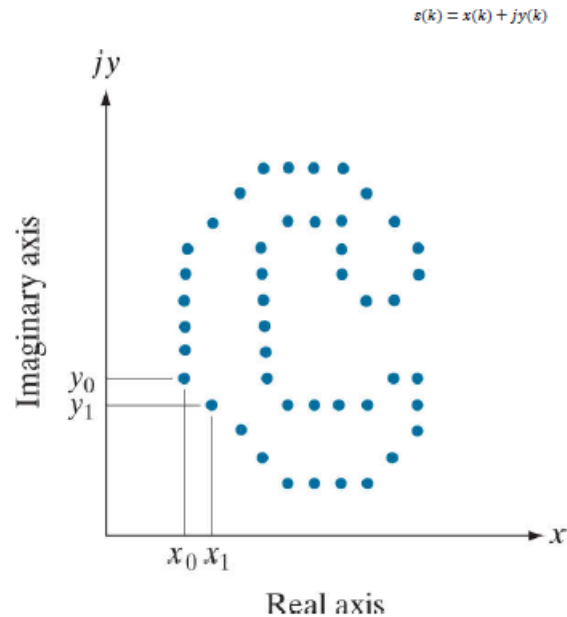
Chain code: 0 0 0 0 3 0 0 3 2 2 3 2 2 2 1 2 1 1

Difference: 3 0 0 0 3 1 0 3 3 0 1 3 0 0 3 1 3 0

Shape no.: 0 0 0 3 1 0 3 3 0 1 3 0 0 3 1 3 0 3

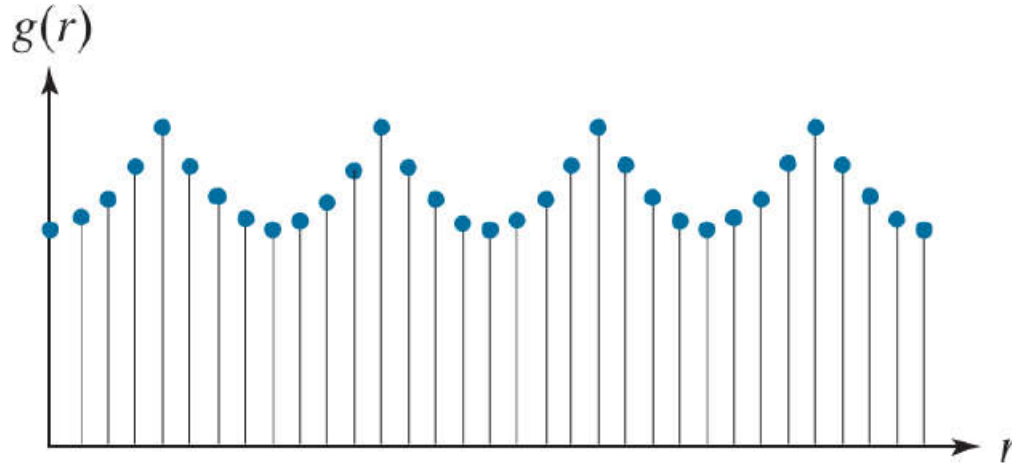
Other boundary Feature Descriptors

► Fourier Descriptors



Other boundary Feature Descriptors

► Statistical Moments





Region Feature Descriptors

Basic Region Descriptors

- ▶ The attributes used in region descriptors are normally **area** which is the number of pixels in the region and **perimeter** which is the length of the boundary
- ▶ One of general region descriptors is **compactness** which can be calculated from

$$\text{compactness} = \frac{p^2}{A}$$

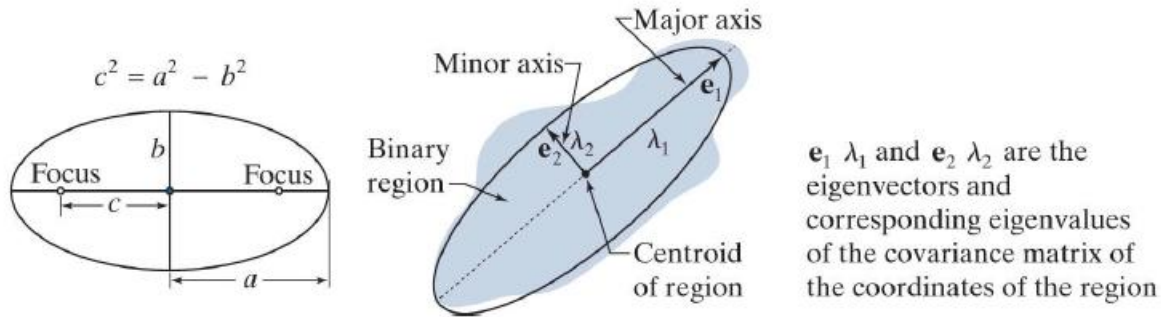
- ▶ Another regular region descriptors is **circularity** which is defined as

$$\text{circularity} = \frac{4\pi A}{p^2}$$

Basic Region Descriptors

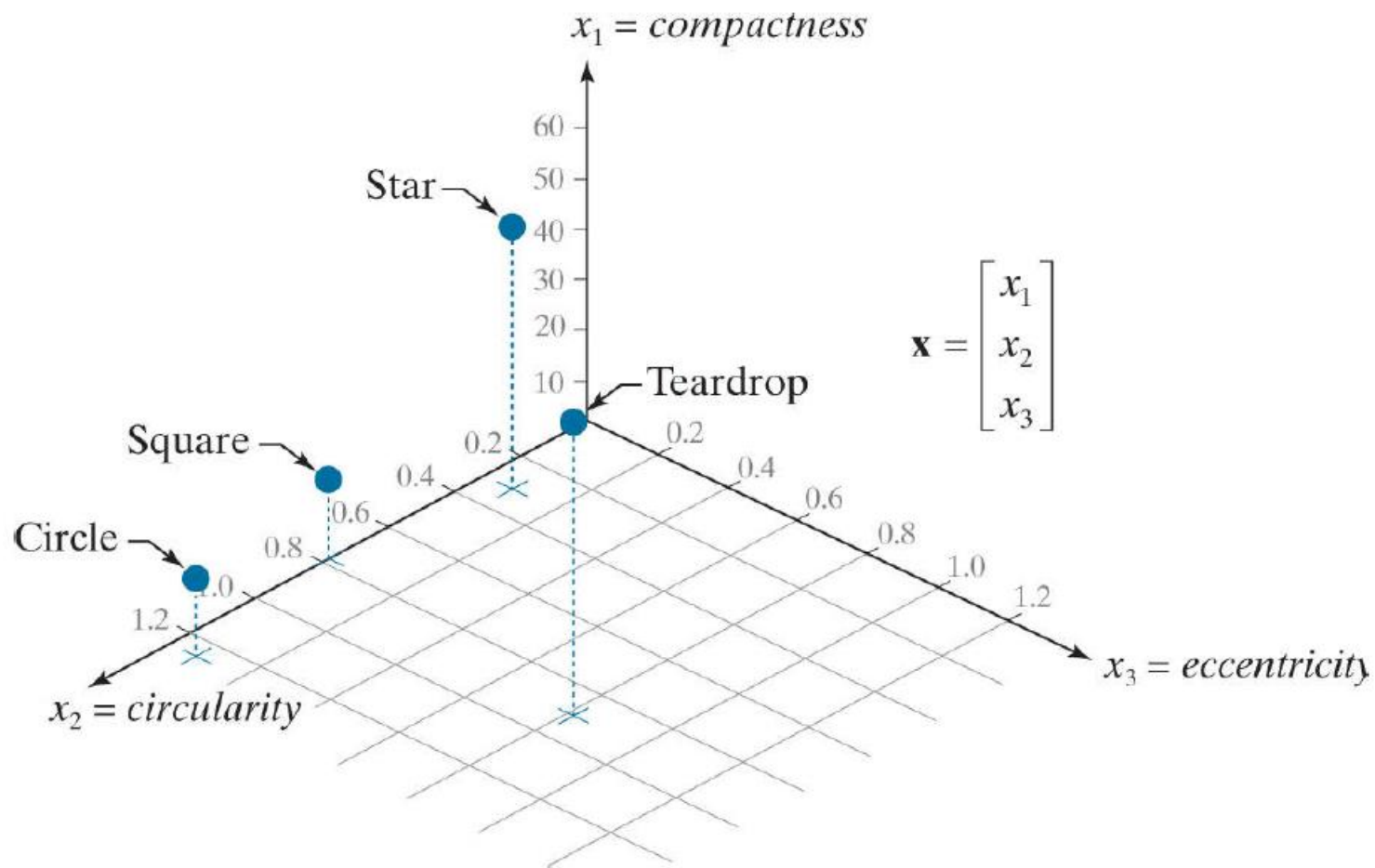
- We can also define eccentricity for the region relative to an ellipse, calculated by

$$\text{eccentricity} = \frac{\sqrt{a^2 - b^2}}{a}, a \geq b$$



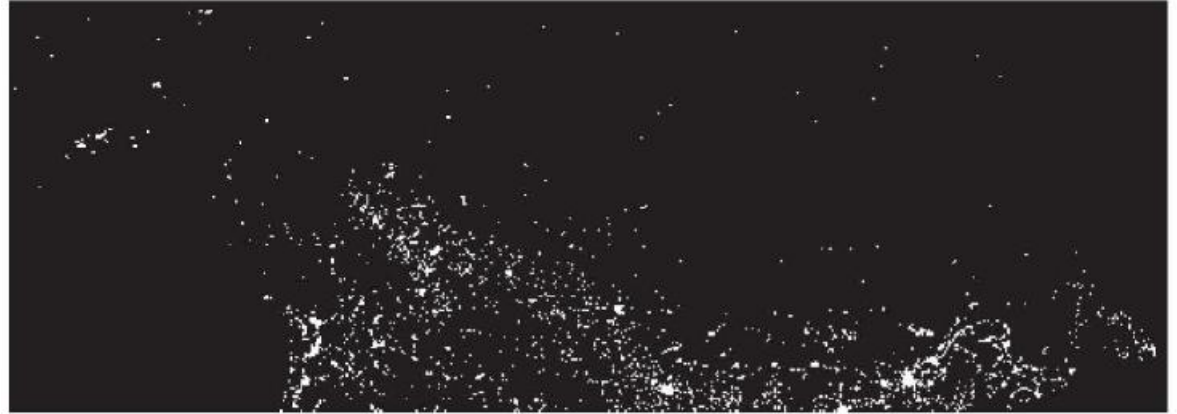
Basic Region Descriptors

Descriptor				
<i>Compactness</i>	10.1701	42.2442	15.9836	13.2308
<i>Circularity</i>	1.2356	0.2975	0.7862	0.9478
<i>Eccentricity</i>	0.0411	0.0636	0	0.8117



Example: Using Area Features

Region no. (from top)	Ratio of lights per region to total lights
1	0.204
2	0.640
3	0.049
4	0.107



Example: Using Area Features

Region no. (from top)	Ratio of lights per region to total lights
--------------------------	---

1	0.204
2	0.640
3	0.049
4	0.107



Example: Using Area Features

Region no. (from top)	Ratio of lights per region to total lights
1	0.204
2	0.640
3	0.049
4	0.107



Region no. (from top)	Ratio of lights per region to total lights
1	0.204
2	0.640
3	0.049
4	0.107



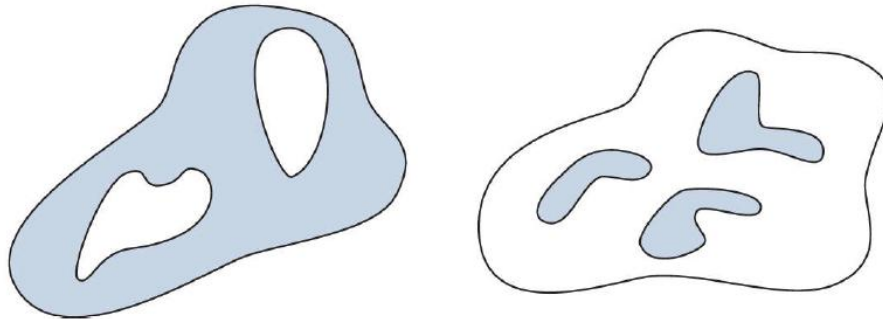


Region no. (from top)	Ratio of lights per region to total lights
1	0.204
2	0.640
3	0.049
4	0.107

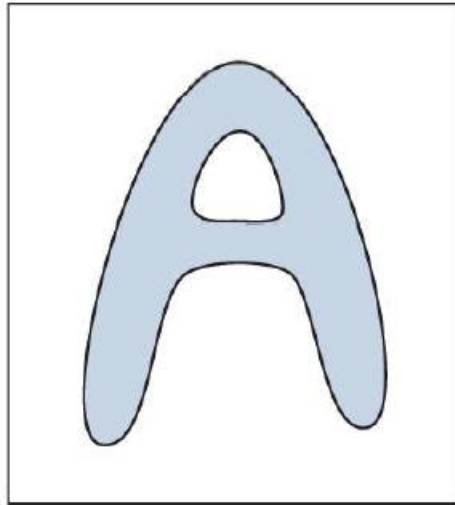
Topological Descriptors

- ▶ Topological descriptors are the number of connected components
- ▶ Let H is the number of holes and C is the connected components, a topological property, called The Euler number is calculated by

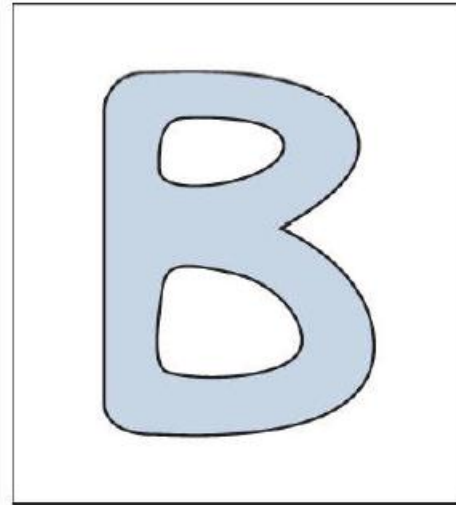
$$E = C - H$$



Topological Descriptors



Euler number = 0

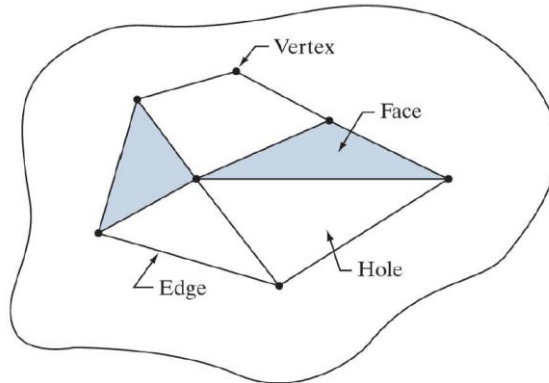


Euler number = -1

Topological Descriptors

- Normally, a straight-line-segment region (polygonal networks) consists of attributes such as number of vertices – V , number of edges – Q , number of faces – F . The relationship of these attributes in Euler formula is

$$V - Q + F = C - H$$

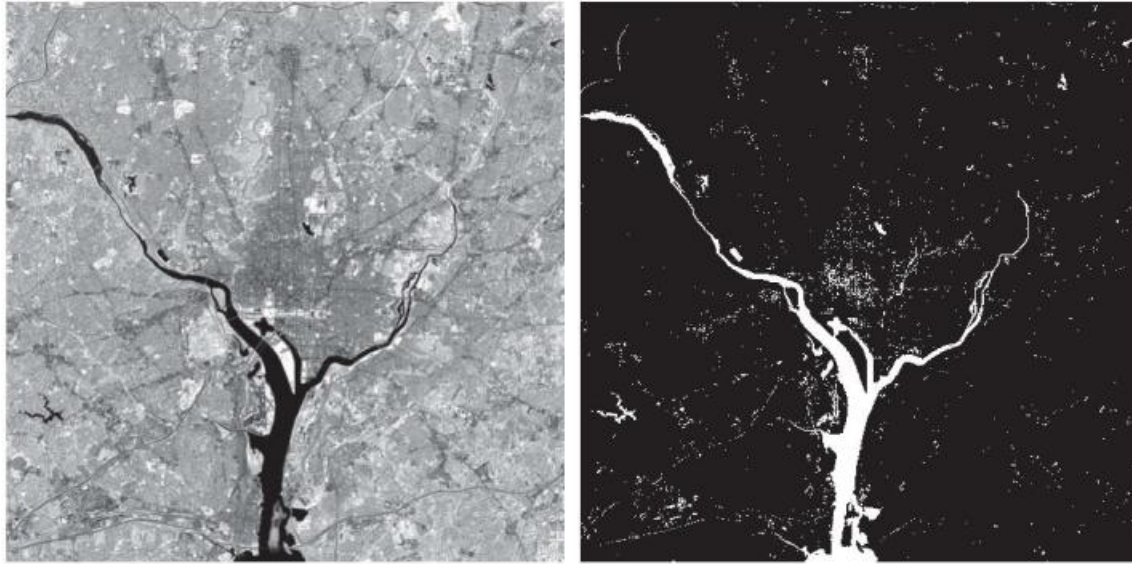


Topological Descriptors

- ▶ Therefore, the Euler number can also be calculated by

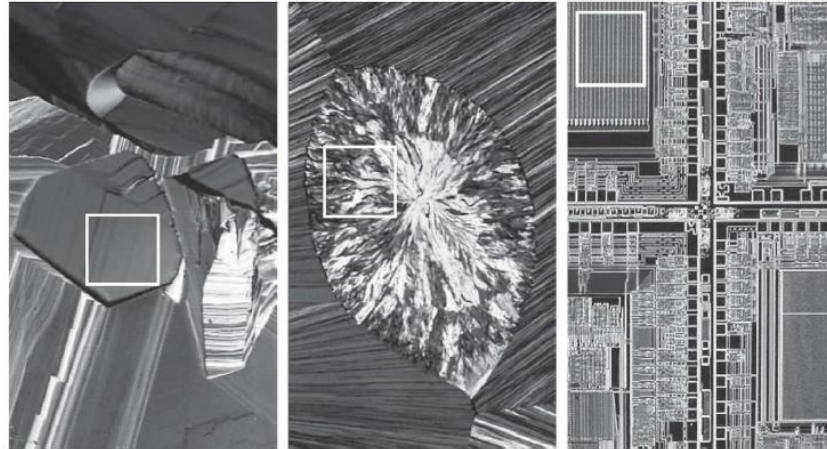
$$V - Q + F = E$$

Topological Descriptors

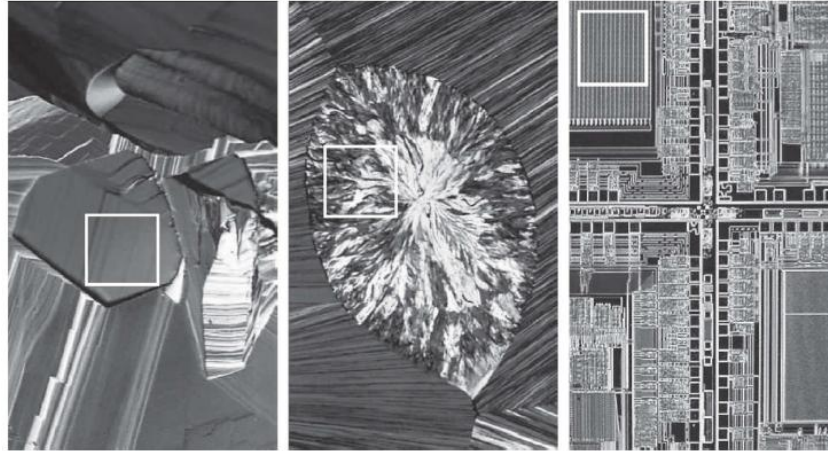


Texture

- ▷ Texture is one important region description. However, there are still no formal definition exists
- ▷ The properties of texture could be smoothness, coarseness, and regularity

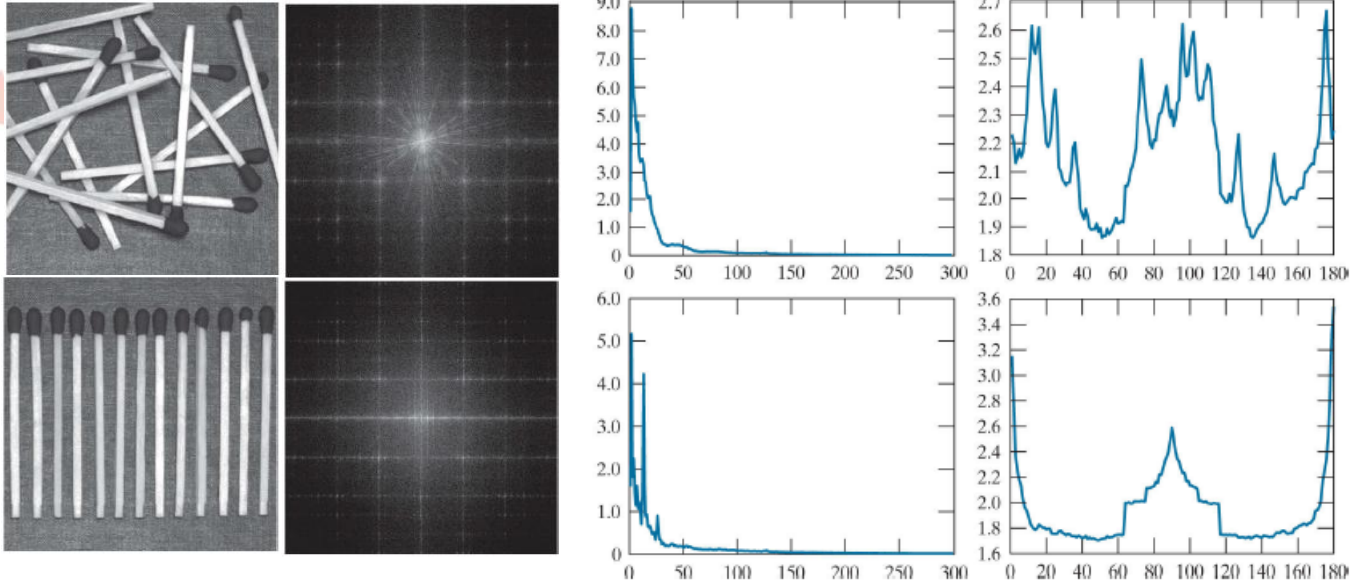


- We can use statistic to approach the texture descriptors such as mean, standard deviation, variance, etc.



Texture	Mean	Standard deviation	<i>R</i> (normalized)	3rd moment	Uniformity	Entropy
Smooth	82.64	11.79	0.002	-0.105	0.026	5.434
Coarse	143.56	74.63	0.079	-0.151	0.005	7.783
Regular	99.72	33.73	0.017	0.750	0.013	6.674

Spectral Approaches



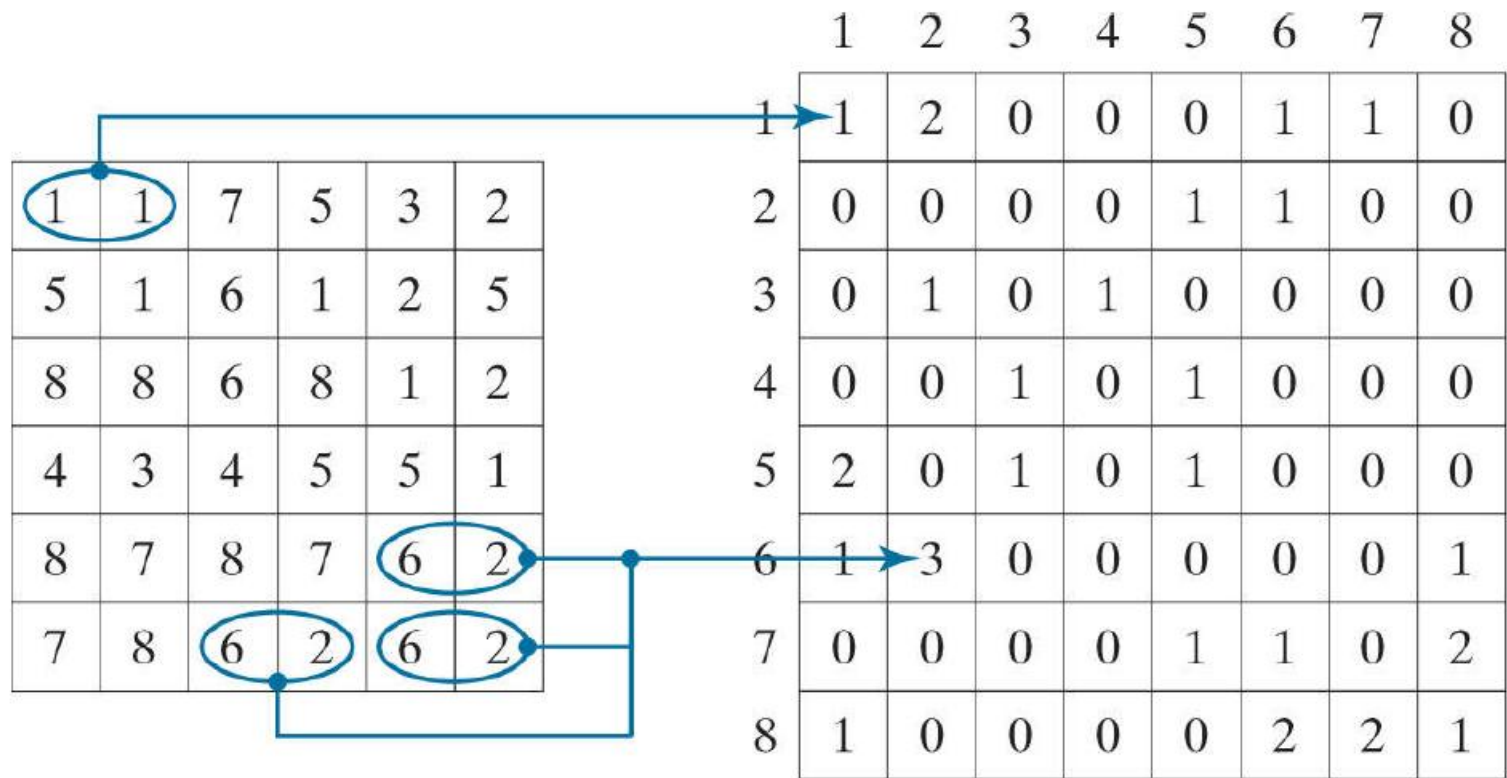
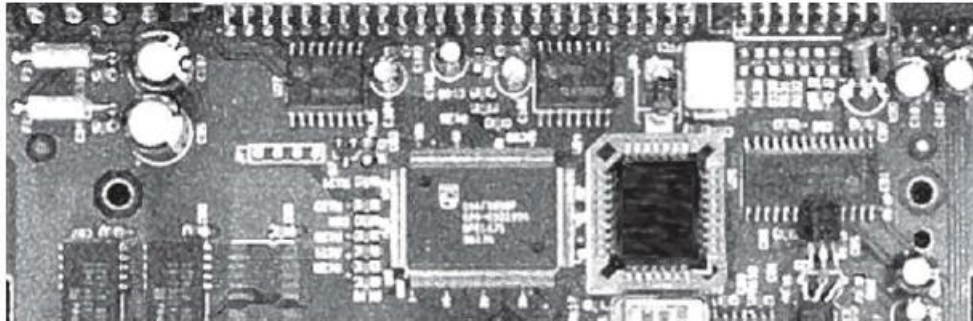
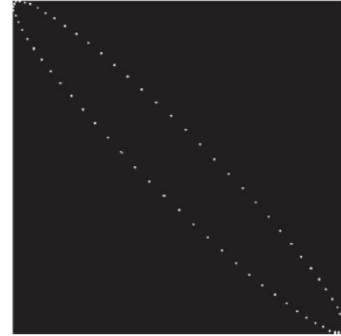
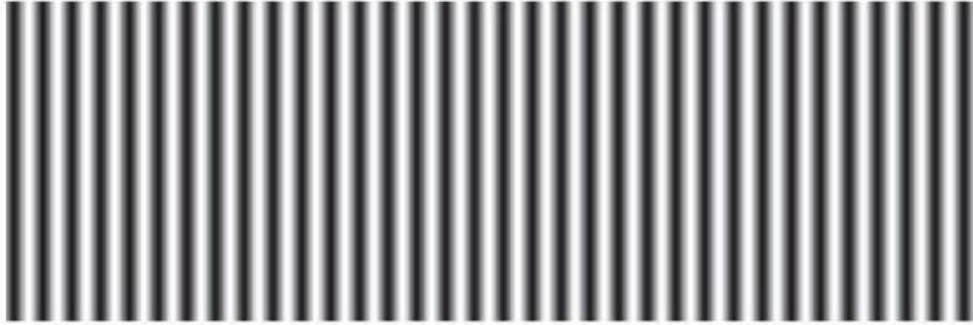
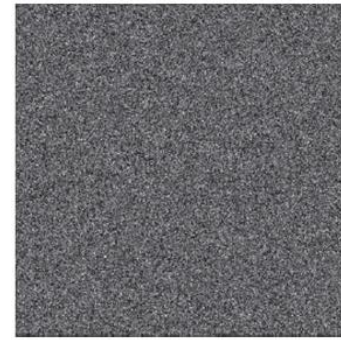
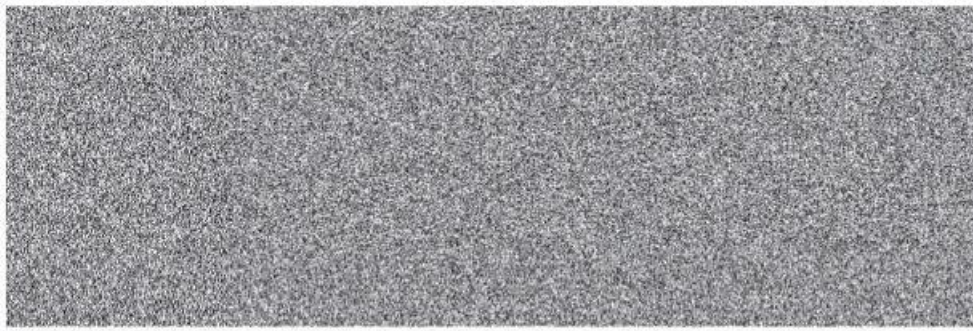


Image f

Co-occurrence matrix G

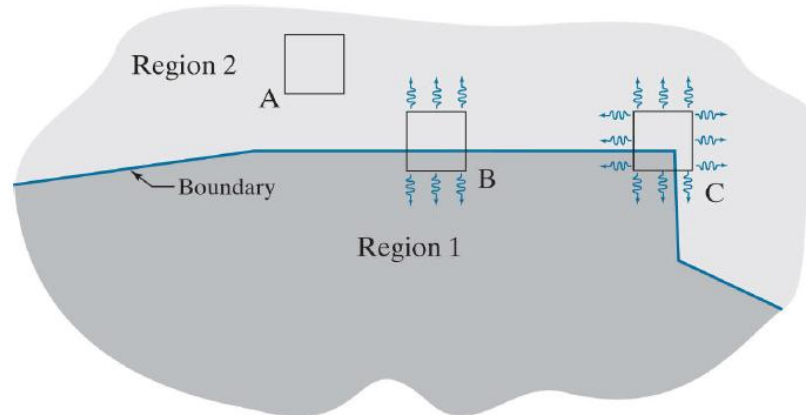




Whole-Image Features

The Harris-Stephens Corner Detector

- ▶ There are three scenarios
 1. Location A – zero or small intensity changes – constant region
 2. Intensities change in one direction – edge or boundary
 3. Intensities change in all direction – corner



The Harris-Stephens Corner Detector

- ▶ Pixels in the window can be written in the matrix form as

$$C(x, y) = [x \ y] \mathbf{M} \begin{bmatrix} x \\ y \end{bmatrix}$$

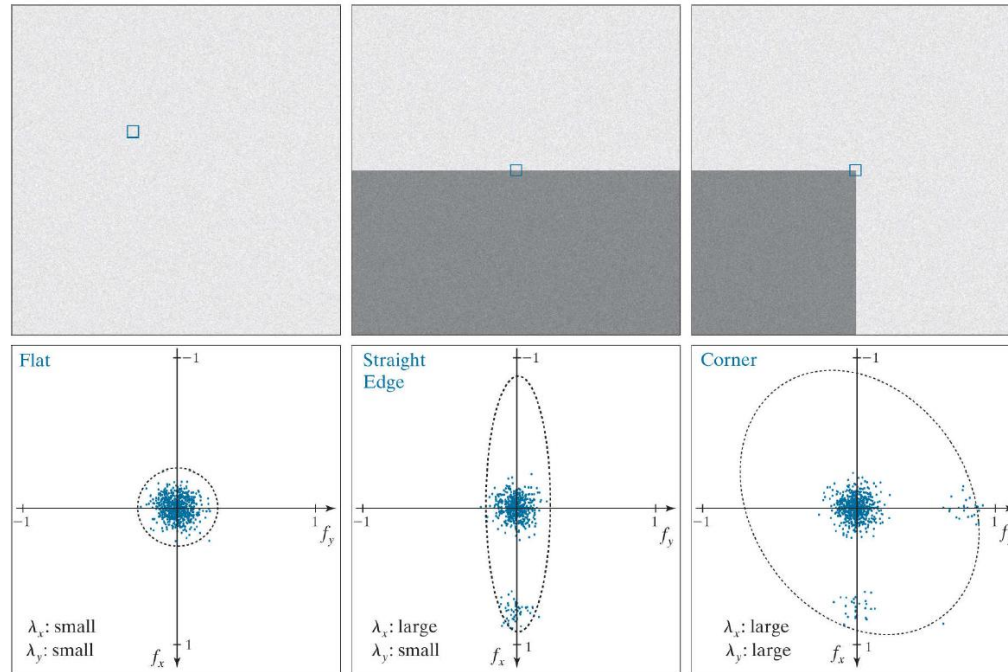
- ▶ Where

$$\mathbf{M} = \sum_s \sum_t w(s, t) \mathbf{A}$$

- ▶ And

$$\mathbf{A} = \begin{bmatrix} f_x^2 & f_x f_y \\ f_x f_y & f_y^2 \end{bmatrix}$$

The Harris-Stephens Corner Detector



The Harris-Stephens Corner Detector



Image from: Gonzalez, R., & Woods, R. (2014). Digital image processing (3rd ed.). New Delhi: Dorling Kindersley.



End of Week 12